

RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Anonymous Author(s)

Abstract

The parts of a language specification that matter most for correctness are its error cases—malformed input, out-of-range values, type errors, division by zero—and these are exactly the branches that random fuzzing reaches slowly and that hand-written conformance suites cover unevenly. We present RuSMT, a framework that derives both a conformance oracle and a directed test generator from a single executable specification. The author writes the semantics once, as an ordinary recursive Rust program in a restricted DSL whose types denote Z3’s theories, and marks each error or specification-violation branch with a named marker such as `Path::named("division_by_zero")`. Compiled, the program is the reference oracle; transpiled to SMT-LIB, it produces one reachability query per marker, and each satisfying model decodes to an input that drives execution to that branch.

We do not assume the solver is complete: the queries combine recursive functions with several theories, so unknown and timeout are expected and cost only coverage. What we make sound is *acceptance*. A candidate is admitted only when Z3 proves that the lifted semantics reach the target marker *and* independent re-execution through the concrete semantics reaches the same marker; an accepted test therefore needs no manual inspection, unlike a fuzzer’s output. This rests on a single obligation we *check* rather than assume—that the transpilation is faithful to the Rust it lifts. When the solver stalls, an untrusted proposer—a bounded enumerator or a language model—may supply a candidate, admitted only through the same Z3-and-replay gate.

We evaluate three semantics of increasing difficulty: an imperative language with dynamic errors, a simply-typed lambda calculus with static type errors, and a TOML 1.1.0 parser with 186 markers. Z3 synthesizes every marker for the first two, under native recursion and depth-bounded unrolling. On the parser, free symbolic search finds no witness within budget (0/186); an array-free re-encoding restores sub-second validation of a determined candidate, so a proposer—in our reported runs a deterministic, model-free stand-in—recovers 151/186 markers through the same gate. Every reported result is model-free and reproducible from the anonymous artifact.

CCS Concepts: • Software and its engineering → Software testing and debugging; Formal software verification; • Theory of computation → Automated reasoning.

Keywords: executable semantics, conformance testing, bounded program synthesis, SMT, Z3, solver-aided programming, Rust, large language models

1 Introduction

A language ecosystem typically carries three artifacts meant to say the same thing: a prose *specification*, an *implementation* adhering to the specification, and a *test generator*. Written separately, they drift. Fuzzing covers the bulk of the input space but misses the narrow error and edge conditions a specification cares about most; a hand-written suite forces its author both to decide which conditions are interesting and to craft inputs exercising them. RuSMT keeps the author’s judgment—expressed as markers in the semantics itself—and hands input generation to a backend solver, so when the specification changes the conformance suite is *regenerated*, not re-authored by hand—a living, executable specification that drives its own testing.

Why error cases. Markers target a language’s *negative space*: its *syntactic* errors, which are ill-formed input the grammar rejects, and its *semantic* errors—well-formed input violating the static or dynamic semantics, such as an out-of-range value, a duplicate key, a type error, or a division by zero—the textbook front-end/analysis boundary [18]. The focus is empirically grounded, not a convenience of what the solver finds easy. Error-handling code is where defects concentrate and testing is thinnest: a majority of *catastrophic* distributed-systems outages trace to incorrect handling of errors, often in code a trivial test would have caught [34], and error paths are routinely dropped in Linux file systems [8]. It is also where implementations *diverge*—no two JSON parsers agree across malformed documents [23], and parser/validator discrepancies on ill-formed input are a documented security-vulnerability source [4, 22]—and what conformance suites assert normatively [6]. RuSMT therefore does not fuzz values broadly; it synthesizes inputs reaching the specification’s own error and violation branches, then differentially tests how implementations treat them [14].

RuSMT collapses two of these three artifacts into one. The author writes the executable semantics *once*, in a restricted dialect of Rust whose standard library’s types denote Z3’s mathematical theories. That single source is given two interpretations: run as ordinary Rust it is the concrete conformance *oracle*; lifted to SMT-LIB it is the source of a solver query that *synthesizes* programs reaching designated points. The author marks a point with a named marker, e.g. `Path::named("division_by_zero")`; RuSMT asks Z3 for

an input driving execution there, decodes the model into runnable source, replays it, and hands the witness to an independent implementation, where a divergence in observable behaviour is a conformance bug.

When the specification changes, the suite is *regenerated* by re-running synthesis against the same markers rather than re-authored by hand. A candidate reaching a marked point may be proposed by anything—the solver, a bounded enumerator, a language model—but it is accepted only after Z3 verifies it against the lifted semantics and re-execution confirms it through the concrete reference. Two assumptions then remain, and we are explicit about them: that the transpilation is faithful to the Rust it lifts, which we *check* per primitive (§5), and that the author’s semantics resembles the prose it formalizes, which we trust, as one trusts any reference implementation.

Research questions. The paper answers four questions. **RQ1:** Can a direct-style Rust interpreter, written in a small SMT-denoting DSL, serve simultaneously as conformance oracle and marker-directed test generator? **RQ2:** What soundness discipline is sufficient for accepting a generated or proposed test without manual inspection? **RQ3:** Where does solver-only synthesis stop on realistic semantics, and can untrusted proposers recover tests while keeping Z3 in the acceptance loop? **RQ4:** Is the approach reproducible enough for artifact review: fixed commands, solver versions, budgets, generated queries, and replay certificates?

Contributions. This is a tool contribution: not a new solver, and not a completeness theorem. **(1) A Rust-native framework anyone can reuse.** You write an ordinary recursive interpreter over the DSL vocabulary and get two artifacts from one source: compiled by `rustc` it is the conformance *oracle* (with Rust’s own type- and memory-safety guarantees), and lowered to SMT-LIB or the Z3 C API it is a marker-directed *test generator*—no separate grammar, no hand-written SMT (§3–4). **(2) Solver-gated acceptance that needs no human inspection.** Named markers are stable obligations, and a witness is kept only when Z3—acting as an automated prover—validates the lifted path condition and concrete replay fires the same marker in isolation (§5). This is the property a fuzzer’s raw output lacks: no one inspects a witness to decide whether it hit the intended case. **(3) An architecture that attacks incompleteness with an untrusted proposer.** Native recursion and depth-bounded unrolling handle what the solver can; on the unknown/timeout frontier an untrusted proposer—a bounded enumerator or a language model—guides the search or proposes whole inputs, and an LLM may even draft the semantics itself, always through the same Z3/replay (or gate) discipline (§6, §9). **(4) Evidence that the DSL is expressive.** We write three semantics that span dynamic (IMP), static (STLC), and real-world parsing (TOML, 186 markers) error spaces; together they show the DSL reaches beyond toy arithmetic and make

the solver frontier visible (§7–8). **(5) A reproducible artifact.** It records the budgets, commands, generated queries, replay certificates, recovered suites, and the partial Z3-backed checks of the DSL primitive contract.

2 Background

SMT as the reasoning backend. RuSMT uses SMT to ask one operational question: *does there exist an input such that this executable semantics reaches marker d ?* The DSL therefore maps Rust expressions to a fixed vocabulary of Z3 theories [5]: booleans, integers/reals, bit-vectors, strings, sequences, sets, arrays, algebraic datatypes, and recursive functions encoded in SMT-LIB 2.6 [2]. For each marker the backend emits a formula of the form

$$\exists x. \text{entry}(x) = \text{Err}(\text{marker_id}(d)),$$

where x is the object-language program or input text. On `sat`, Z3 returns a model for x ; on `unsat`, no such x exists *for that encoding and bound*; on `unknown` or `timeout`, the framework learns only that this backend did not finish.

The scientific and engineering issue is that the generated formulas are outside a decidable quantifier-free fragment. A recursive interpreter becomes `define-funs-rec`; Z3 reasons about such definitions by unfolding and heuristics. Parsers additionally combine strings/sequences, arithmetic, bit-vectors, recursive datatypes, and occasional quantifiers handled by incomplete instantiation. Thus RuSMT treats Z3 as a *sound arbiter of returned verdicts*, not as a complete search oracle: `sat` candidates must still replay through the Rust semantics, and `unknown/timeout` costs only coverage. The same split explains why proposer recovery is useful. Searching for an unconstrained input may be intractable, while validating a fully determined candidate can be easy because the backend macro-inlines the concrete sequence and Z3 constant-folds along one path.

Executable semantics and synthesis. A *semantics* fixes the meaning of a language; it is *executable* when that definition is itself a runnable artifact—a reference interpreter written *to be* the specification, not merely any implementation that happens to run programs (a production compiler runs the language without being its definition). Frameworks like K [20], PLT Redex [7], and Ott/Lem [16, 24] write that definition as rewrite/inference rules and derive a tool suite from them. RuSMT instead takes the semantics to be an *ordinary recursive function*: the same source executes concretely as the oracle and compiles symbolically as the search space. Compared with fuzzers such as Csmith [33], the target is not coverage in general but a named specification obligation. Compared with SyGuS and SemGuS [1, 10], the user does not author a separate grammar or CHC semantics; the unknown is an input to the interpreter the user already wrote.

3 The RuSMT DSL

A RuSMT specification is an ordinary Rust program, written once and read two ways: compiled by `rustc` it *runs* as the reference interpreter; lifted by the transpiler it *denotes* an SMT formula. The design principle is that these two readings must agree by construction, so every construct admitted into the DSL is one with a mathematical meaning the solver can express. That single requirement dictates the whole subset: RuSMT is a deliberately *functional*, *total* fragment of Rust.

A functional subset, and why each restriction earns its place. The DSL keeps immutable `let` bindings, `if` with a mandatory `else`, `struct/enum` with `match`, and (mutually) recursive calls; it forbids loops, mutation, and references. These are not arbitrary—each exclusion buys a property the lift depends on. *No mutation or references* means a value never aliases or changes under another name, so a `let` is mathematical substitution and the translation is referentially transparent; this is the same discipline that lets a big-step operational semantics be read as a system of equations. *No loops* leaves recursion and (bounded) quantifiers as the sole means of iteration, matching how a solver expresses repetition—as recursive definitions and quantified formulas, not as mutable state evolving over time. The payoff is that the interpreter reads like the inference rules it formalizes: Winskel’s big-step rules for IMP (§7.1) transcribe almost line for line, and there is no gap between “the specification” and “the code that runs.”

Totality makes error cases first-class. The subset is, crucially, *total and exception-free*: no `panic!`, no `?`, no `.unwrap()`, no early return. A partial function has no total SMT reading, so any divergence from the happy path must be expressed as a returned *value*—for example a `ParseResult::Err` carrying a `Path` (below)—rather than a thrown exception. This is exactly what the negative-space thesis (§1) wants, and it comes for free: every error condition is, by construction, an explicit branch returning an error value, i.e. a place a marker can sit, on code that must exist anyway.

Separation of concerns. Two layers are kept strictly apart. The *standard library* (below) is a fixed vocabulary denoting $Z3$ ’s theories and nothing else; the *interpreter* encodes a *target* language’s semantics on top of that vocabulary. A target’s idiosyncrasies—two’s-complement wraparound, a parser’s error conditions, an uninitialized-variable rule—live in the interpreter as explicit guards, never inside the `stdlib`. This keeps the per-construct soundness obligation (§5) small, fixed, and language-independent: it is checked once for the `stdlib`, not re-argued per case study.

Mechanical enforcement. Two attribute macros enforce the subset at compile time, so an out-of-subset specification fails to build rather than mistranslating

silently. `#[smt_type]` admits only `structs` and `enums` whose every value is `Copy` (so nothing aliases); `#[smt_fn]` rejects `const/async/unsafe` and variadic functions, method receivers, and any body beyond `let` bindings and a trailing expression.

Listing 1 shows the whole discipline in one IMP fragment—the division guard of the arithmetic evaluator. It is ordinary recursive Rust; the only synthesis-specific token is `Path::named`, sitting on the error branch a total evaluator must have anyway. The primitive `bv_div` already agrees with $Z3$ on every nonzero divisor, so the interpreter guards only the diverging case (a zero divisor) and falls through to `bv_div` otherwise—the guard/if-then-else pattern of §5. From this single source RuSMT synthesizes `A := (0 / 0)` (§8); replayed through the same `eval_aexp`, it reaches the marker.

```
if *new_r.eq(I64::from(0)) {
    ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
    ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Listing 1. The division guard of IMP’s evaluator (operands already evaluated to `new_l`, `new_r`; `Err` propagation elided).

Standard library: the theory vocabulary. The DSL’s types denote $Z3$ sorts: `Boolean`, unbounded `Integer/Real` (big integers/rationals), the bit-vectors `I32/I64/U32/U64` (the signed/unsigned split is a DSL-API reading of the same $Z3$ sort), `IEEE F32/F64`, `String` (the Strings theory), `Seq<T>`, `Set<T>`, and `Array<K,V>`. As established above, these denote $Z3$ ’s theories, not Rust’s native semantics and not any target language’s. Two helper types are frontend-only. `Cloak<T>` is a Box-like indirection that lets an author write self-referential ADTs (e.g. `Add(Cloak<Aexp>, Cloak<Aexp>)`) despite Rust’s sized-type rule; the IR erases it entirely—every `Cloak<T>` field lowers to `T` and shield/reveal to identity—since $Z3$ supports recursive datatypes natively (the one proviso is well-foundedness: a `Cloak<T>` must co-occur with a non-recursive variant, so the lowered datatype is inhabited). `Path`, the marker type, is described next.

Markers via Path. A marker flags a program point the author wants covered—characteristically an *error or specification-violation* branch (the “negative space” of §1); markers are *syntactic* or *semantic* (the taxonomy above), and the three case studies span both. The value a marker carries is a `Path`, and its *identity* is what lets the marker be matched across the two readings of the specification. An id drawn from a global counter would be order-dependent, and so unmatchable against a replayed run; instead `Path::named("d")` derives its id as a fixed hash of the name `d`, and the *same* pure function serves transpiler and evaluator, so the id the query asserts and the id a re-executed witness carries are *identical by construction*.

In the SMT backend a Path is represented by that id as a plain Integer (with a sentinel for “no marker reached”). The choice is deliberate: an integer equality remains decidable through the deep recursive unfolding of a lifted interpreter, whereas a set-membership test would force the solver to construct and probe a symbolic collection it cannot evaluate on those paths—an encoding decision we return to in §4. Each named marker is thus one synthesis target (§6); named markers are what make per-target certification sound (§5), and the case studies we replay use them.

Quantifiers. forall!, exists!, and choose! exist in a *bounded* form over a collection (forall!(x in xs => p)). choose! is Hilbert choice: concretely the first satisfying witness, in SMT a Skolem function under the standard choose axiom—sound because nothing may depend on *which* witness is chosen, only that it satisfies the predicate.

4 The compilation pipeline

The same annotated Rust source is compiled in two modes. In oracle mode, rustc executes the interpreter on concrete candidate inputs. In solver mode, the transpiler parses the restricted Rust subset, lowers it to a sort-checked IR by Hindley–Milner unification over SMT sorts, monomorphizes generic functions on demand, records each Path::named marker, and emits a per-marker query. Listing 2 shows the backend loop. Every query is inspectable SMT-LIB2, which is what makes each reported result re-runnable with a bare z3 -smt2 on a committed file.

```

360 for each named marker d in entry:
361   Q = lower_to_smtlib(entry, marker_id(d), k)
362   r = run("z3 -smt2 -t:<budget_ms>", Q.main)
363   if r == sat:
364     c = decode_model(r.model)
365     if replay(c, d) == ReachedTarget:
366       accept(c)
367     else:
368       report_replay_rejection(d, c)
369       continue
370   if r == unsat:
371     report_unreachable_under_encoding_or_bound(d)
372     continue
373   // Reported recovery path: only after unknown/timeout.
374   if r notin {unknown, timeout} or proposer is disabled:
375     report_unreached(d, r)
376     continue
377   for round in 1..R:
378     if mode == guided:
379       S = proposer.scaffold(d, history)
380       r = run_z3(Q.main + assertions(S), guide_budget)
381       if r != sat:
382         feedback(r) // relax on unsat, tighten on
383         timeout
384         continue
385       c = decode_model(r.model)
386     else:
387       c = proposer.whole_input(d, history)
388   v = run_z3(macro_inline(Q.validation, c), budget)
389   if v == sat && replay(c, d) == ReachedTarget:
390     accept(c)
391     break
392   feedback(v, replay(c, d))

```

Listing 2. Backend loop for one entry function. Queries are SMT-LIB2 text; the guided loop of §9 keeps one z3 process alive per target and scopes each round with push/pop, so the base theory is parsed once and an extra model costs only a blocking clause.

Backend and budgets. The backend renders inspectable .smt2 files from the IR and drives z3 as a subprocess. Independent checks are one-shot; the guided loop of §9 instead holds a single z3 -in process open per target and scopes each round with push/pop. That choice is what makes the loop affordable: the base theory is parsed once rather than once per round (measured at ≈ 0.9 s for the 206 KB TOML base), the solver’s state survives between rounds, and a *further* model of the same round costs only a blocking clause and a re-check instead of a fresh solve. The rendered per-round query is still written out, so nothing becomes less inspectable. The default backend timeout is 600 s; the reported TOML search frontier uses RUSMT_BACKEND_TIMEOUT_SECS=20. Direct proposer recovery tries at most RUSMT_LLM_MAX_GUESSES inputs (default 4). Guided recovery uses RUSMT_GUIDE_ROUNDS scaffold rounds (default 4), a per-round Z3 budget RUSMT_GUIDE_Z3_SECS (default 30 s), and up to RUSMT_GUIDE_MODELS distinct models per round (default 3). Authoring gates use RUSMT_AUTHOR_Z3_SECS (default 20 s) and four drafting rounds unless overridden. These are hyperparameters of the artifact, not tuned per result: IMP uses native recursion (k=0), STLC uses k=4, and the TOML pure-search sweep uses k=0.

Engineering for Z3. Several encodings are deliberately solver-facing. A marker is an Integer id, not a symbolic set membership test, because equality survives deep recursive unfolding. Recursive maps such as TOML tables and STLC environments are encoded as array-free inductive association lists; an Array whose range is a recursive datatype made even concrete-candidate validation return unknown in early TOML experiments. Recursion is offered either as native define-funs-rec or as fixed-depth unrolling because neither dominates across the case studies. The backend parses the leading Z3 status line before trusting process exit status, since Z3 can exit nonzero after benign model-output failures. A sat model is then rendered back to object-language source by re-sharing let terms, unmangling constructors, canonicalizing solver-chosen names, and decoding bit-vectors as two’s-complement integers.

5 Soundness of the lift

The trusted computing base. The trusted base is the RuSMT specification together with the transpiler (the lift to SMT) and Z3. We *assume* the prose-to-DSL transcription correct (trusted, not verified), and we trust the transpiler

and solver *because* the lift is faithful—the per-construct contract below, *checked* rather than asserted—so trusting Z3’s verdict reduces to that one obligation. The concrete parser and pretty-printer are also trusted; a bug in any is a bug in the results. Replay is not what discharges this trust: it is a per-candidate cross-check that the symbolic and concrete readings agree, catching a faithfulness defect on any candidate it touches as a spurious rejection. Per-target certification additionally assumes `marker_id` is injective over the (finite, author-declared) set of marker names—as a 63-bit hash, a checkable side condition trivially met by the handful of names we use.

The per-construct contract. Soundness rests on a contract for every standard-library primitive:

Definition 5.1 (Per-construct soundness contract). For every stdlib function f and every concrete input x ,

$$\text{rust_impl}(f, x) = \text{z3_eval}(\text{z3_formula}(f), x).$$

The stdlib denotes Z3’s theories; bridging it to a given target language is the interpreter author’s job.

The guard / if-then-else pattern. A primitive matching the target language is used directly; one that diverges only on specific inputs is *guarded* on those inputs, falling through to the (Z3-correct) primitive otherwise. Listing 1 has this shape: `bv_div` agrees with Z3 everywhere, so the only special case is a zero divisor, which the guard turns into a marker. Because concrete Rust and Z3 take the *same* branch on the same input, Definition 5.1’s equality survives the conditional—and the same discipline appears inside the stdlib (e.g. a width-or-more shift returns 0 to match Z3’s `bvshl`).

How the contract is validated. Validation is layered: (i) each stdlib method’s doc-comment states the exact Z3 formula it must match; (ii) example-based unit tests pin the Rust side to Z3’s documented result and have caught real divergences; (iii) for the error-prone *bit-vector* families (signed vs. unsigned division/comparison; over-width shifts; `i64::MIN / -1`) we discharge the contract *empirically* with a randomized differential harness comparing `rust_impl` against the *backend-emitted* formula (§8); the cross-backend both mode is an extra whole-query check that catches pipeline regressions.

Single free input; fixed context. A load-bearing point governs *which* inputs are free in the synthesis query.

Principle 1 (Free/fixed agreement). The set of inputs treated as free by the SMT query must coincide with the set of inputs left free by the concrete oracle; everything else must be fixed to the same value on both sides.

The IMP entry point `eval_com(c: Com)` fixes the empty store internally, so the program is the single free input; were the store left free, Z3 could choose a starting state the

concrete run never uses—a “witness” that does not replay. TOML’s `parse_toml` needs no wrapper.

Acceptance by Z3, cross-checked by replay.

Observation 1 (Double-gated acceptance). A candidate is accepted for target d only if Z3 *validates* that the lifted semantics reach the *named* marker d , *and* the same candidate, re-executed through the concrete interpreter, fires the *same* d —not merely some marker.

Z3 validation is the verification step; replay is an *independent* cross-check of the identical reachability fact through the concrete reading of the spec, enforced by a small certifier (§9) that re-parses and re-executes the decoded program under a wall-clock bound and checks membership of the *same* `marker_id(d)` the query asserted—so a candidate firing a *different* marker is rejected. The cross-check is load-bearing for two reasons. First, it *guards against* a violation of the lift obligation (Definition 5.1) on whatever constructs the candidate exercises: were the transpilation unfaithful on one, Z3 could validate a candidate the concrete semantics do not, and replay catches exactly that disagreement (the known stdlib divergence `Array::contains_key` on sentinel values thus causes only a *spurious rejection*, never a false accept). Second, quantified or depth-bounded queries can return a *candidate* model (Z3 reports `sat` with `(:reason-unknown "")`) that does not genuinely reach the marker; replay rejects these too, and §8 shows the catch firing on a real example. Identifier and bit-vector decoding do not affect which marker fires.

Completeness: not claimed. We do *not* claim completeness in general. Even when an input reaching a marker exists, Z3 may return unknown or time out: the encoded formula can fall outside what the solver decides within its resource limits. At best we have *bounded* completeness—relative to a structural/fuel bound (§6)—and even that is contingent on the solver.

6 Bounded synthesis

Markers to queries. For a target marker with id i ($i = \text{marker_id}(d)$ for a named marker d), the query declares one constant per top-level parameter (`input_0, \dots`), applies the entry function, and asserts that the returned error value’s carried Path *equals* i (an integer equality, guarded by the enum constructor test that selects the error variant), then `(check-sat)/(get-model)`.

Bounded recursion by unrolling. A recursive interpreter is not a finite formula. A $k=N$ flag selects between two routes. With $N=0$ (default) recursive strongly-connected components (SCCs) are emitted as Z3 `define-funs-rec` and the solver handles recursion natively. With $N \geq 1$ every recursive SCC is unrolled into $N+1$ depth-indexed copies $f_0, \dots, f_N$: f_d calls f_{d-1} for in-SCC callees, f_0 is a terminator,

and external callers route through f_N . A sat model under depth- N unrolling is a witness reachable in $\leq N$ recursive steps—the sense in which completeness is *bounded*, as in bounded model checking [3].

A non-error terminator makes unrolling productive. The terminator f_0 matters. When the return type has no nullary variant, the backend falls back to an *unconstrained* null sentinel, which the solver may instantiate as the very error value a marker assertion demands, so a depth- N sat is satisfied *through the cutoff* rather than a genuine path. Replay rejects such candidates (we observe it on IMP and STLC, §8), so soundness is never at risk, but the budget is wasted. The remedy is a nullary *bottom* variant never produced by the concrete semantics (so they are unchanged): the cutoff can no longer discharge an `Err`-marker assertion, forcing a real marker-firing path within depth N . STLC needs exactly this—adding it turns depth-4 unrolling from all-spurious into genuine, replay-certified witnesses for every marker (§8). The lesson is general: bounded model checking of a marked interpreter wants a “ran-out-of-fuel” value distinct from every marked outcome.

Model decoding and the loop. A sat model is decoded to runnable source by a printer that extracts `input_0`, inlines let-shared subterms, maps mangled constructors back to AST nodes, canonicalizes solver-chosen identifiers (sound: which marker fires is spelling-independent), and decodes bit-vector literals as two’s-complement signed integers. The loop per target—build, solve, decode, replay, accept only if the marker fires—then runs the witness against an independent implementation, where a behavioural divergence is a conformance bug.

7 Case studies

RuSMT is meant for authors of new executable semantics, so the case studies are selected to stress different authoring and solving regimes rather than to maximize benchmark count. IMP is the small dynamic-semantics baseline; STLC tests static semantics and recursive type structure; TOML tests a realistic parser with both syntactic and semantic errors. In the first two, accepted programs are synthesized by Z3. In TOML, reported recovered witnesses come from a deterministic candidate table standing in for an untrusted proposer; Z3 validates each candidate and replay certifies it.

7.1 IMP: closing the loop end-to-end

IMP is the canonical small imperative “while” language of Winskel [30], chosen because its big-step semantics is familiar and recursive but small enough that native Z3 recursion should succeed if the pipeline is sound. Its grammar has arithmetic and boolean expressions, assignment, sequencing, conditionals, and while loops. We add two dynamic error branches to make conformance

obligations explicit: division by zero and reading an uninitialized variable from the initially empty store. The target programs Z3 produces are correspondingly interpretable: $A := (0 / 0)$ for `division_by_zero` and $A := v0$ for `uninitialized_variable`. This study answers whether a direct-style interpreter can be lifted, solved, decoded, and replayed without bounded unrolling or proposer help.

7.2 STLC: static semantics via bounded unrolling

STLC is chosen to move from dynamic execution errors to static semantic errors. The language has types $T ::= \text{Int} \mid \text{Bool} \mid T \rightarrow T$ and terms for literals, variables, annotated abstractions, application, let, conditionals, arithmetic, and equality. The checker returns either a type or one of eight named type-error markers: unbound variable, non-integer + operands, equality type mismatch, non-boolean if condition, mismatched if branches, application of a non-function, and argument/parameter mismatch. This forces synthesis through a recursive typing environment and recursive arrow types, which native recursion does not solve in budget. With bounded unrolling and a non-error bottom terminator, Z3 synthesizes all markers; for example, the argument-mismatch witness applies a function to an argument of the wrong type, and the `if_cond_not_bool` witness places an integer-valued variable in the condition position.

7.3 TOML: reaching the solver frontier

TOML 1.1.0 [19] is chosen because it is large enough to expose the solver frontier while still having a precise public specification. The executable parser has a single free input, a sequence of Unicode code points, and 151 parser functions covering documents, key-value pairs, tables, arrays, inline tables, booleans, integers, floats, strings, and datetimes. Its 186 named markers cover syntactic errors (malformed keys, unterminated strings, invalid escapes, missing array/table delimiters) and semantic violations (integer/float overflow, out-of-range date/time fields, duplicate keys, and redefined tables). Representative recovered candidates are small and human-readable, e.g. `key = FALSE` for the case-sensitive boolean marker and `key = 1997-00-25` for the month-range marker. We do not claim a verified complete TOML formalization; the study asks whether a substantial executable parser can still be used as an oracle and whether Z3 can validate proposer candidates when full search fails.

8 Evaluation and discussion

Evaluation questions. The evaluation follows the RQs from §1: whether one executable semantics can be both oracle and synthesis source (RQ1), whether acceptance can avoid manual inspection (RQ2), where solver-only search fails and proposer validation helps (RQ3), and whether the artifact can reproduce the paper’s claims (RQ4).

Table 1. Summary of the main reported checks. All accepted witnesses are validated by Z3 and replay-certified unless noted as pure search.

Check	Setting	Result
Stdlib contract	BV, String/Seq, edge	2715+5317+216 Z3-backed checks, 0 disagreements
IMP synthesis	$k = 0$	2/2 markers sat; both replay-certified
IMP depth bounding	$k = 1$	a $k=0$ -reachable marker returns unsat: bounded unsat $\neq$ unreachable
STLC synthesis	$k = 4$, text	all 8 named type-error markers certified (9 targets)
TOML pure search	186 markers, 20 s each	0 sat, 0 unsat; all targets time out
TOML proposer recovery	array-free, fixed candidates	sub-second sat validation; 151/186 markers certified; map covers 182/186

Setup. We measured on an Apple M1 (8-core, 16 GB) running macOS 26.5 with Z3 4.15.4, driven as a subprocess over SMT-LIB2. IMP synthesis ran for both markers under native recursion ($k=0$) and depth-1/2/3 unrolling; STLC under depth-4 unrolling ($k=4$); the full TOML sweep covered all 186 targets at a 20 s/target budget. Every reported witness is replayed; all numbers are a single run. The differential harness, the synthesis artifacts, and both AI-integration modules are in the artifact.

Reproducibility. Two properties make the *AI-integration* results reproducible—the usual sticking point for an AI-in-the-loop paper. First, every reported number is *deterministic and model-free*: no language model is invoked for any result, so the TOML recovery reproduces from a fixed, artifact-included candidate table with *no API key, network, or nondeterministic call*—the AI integrations are exercised by scripted proposers standing in for an untrusted model. Second, each claimed quantity maps to one committed command; the differential harnesses shell out to the real z3 on the *backend-emitted* formula, and every query is content-addressed by SHA-256. We pin Z3 4.15.4, the toml crate v0.8.23, and CPython ≥ 3.11 ; the anonymous review artifact is public at <https://anonymous.4open.science/r/rusmart-5278/>.

Contract-check results. Before the synthesis results, we confirm the lift itself: we ran the differential harness of §5 on the 64-bit bit-vector families against Z3 4.15.4: 13 signed (I64) operations and the 6 unsigned (U64) operations whose formula differs from the signed one, comparing *rust_impl* against the backend-emitted formula on directed edge cases (including `i64 : MIN / -1` and shift amounts ≥ 64) and seeded random inputs—2715 (operation, input) pairs, with *zero* disagreements, exactly the signed/unsigned split (`bvshl` vs. `bvdiv`, `bvshl` vs. `bvult`) most prone to Rust/Z3 divergence. A companion harness covers the *string/sequence* families (ASCII String, Unicode via `Seq<U32>`): 5317 checks, again

zero disagreements, and a directed edge-class harness adds a further 216. This is *not* the full stdlib contract: it does not yet cover the 32-bit bit-vector variants, reals, or floats—those remain on documented-formula-plus-example-test evidence, extending the harness the main remaining validation work.

IMP: the loop closes (native recursion). Under native recursion ($k=0$) the solver returns a model for both markers and every witness replays; solve times are 0.2–0.6 s (`B := (0 / 0)` in 612 ms, `v0 := v0` in 206 ms). Both witnesses are assembled by the solver from the datatype constructors alone, no IMP fragment supplied: `B := (0 / 0)` reproduces the guard of Listing 1, and `v0 := v0` reads a variable the empty store never wrote.

Depth bounding is delicate, in both directions (measured). Unrolling trades completeness for tractability, and IMP exhibits the trade in both directions. *Bounding can hide a witness*: at $k=1$ the division-by-zero target returns unsat in 205 ms—yet the very same target is sat with a replay-certified witness at $k=0$. A bounded unsat therefore means only “no witness within depth k ” and must never be reported as unreachability; the pipeline enforces exactly this, treating unsat as a genuine verdict only at $k=0$ and otherwise routing the target to the recovery path of §9. *Bounding can also manufacture one*: unrolling lets a model satisfy the marker assertion through the cutoff terminator rather than a genuine path, which is what the STLC terminator experiment below isolates—without the bottom variant, depth-3/4 unrolling yields only cutoff-spurious models that replay rejects. Neither failure is detectable from the solver’s verdict alone, which is why replay is not optional. (At $k=2$ and $k=3$ both IMP targets exhaust the per-target budget, so only the $k=0$ artifacts are included.)

STLC: bounded unrolling with a non-error terminator (measured). The STLC checker (§7.2) is genuinely recursive over three datatypes with structural type equality, and native-recursion *search* ($k=0$) does not return within the per-target budget—placing STLC between IMP ($k=0$ solves) and TOML (out of reach). Under depth-4 unrolling with the bottom terminator of §6, the solver synthesizes a witness for every one of the eight named semantic markers; each is replayed through the concrete `type_check` and fires its target (9 targets in all—“apply a non-function” is reached from two match arms—every one certified), in 5.4–6.5 s. The terminator is load-bearing: *without* the bottom variant, depth-3/4 unrolling returns only cutoff-spurious models that replay rejects (e.g. a deeply nested `let` that type-checks cleanly); *with* it the same depth yields genuine, certified witnesses—including a *higher-order* one for the argument-mismatch marker (an applied λ whose argument type differs from its parameter).

TOML: the solver frontier (measured). Lifting the TOML parser produces SMT that Z3 accepts without error,

but the per-target queries overwhelmingly exceed even a modest solving budget. Across the full sweep of all 186 targets (one per named marker reached in `parse_toml`; 20 s per-target budget, Apple M1, Z3 4.15.4), *no target produced a witness*: all 186 *time out* (0 *sat*, 0 *unsat*, 0 unknown, 0 backend error). Under the array-free encoding the theory-incompleteness unknowns and backend errors of the original array encoding are gone, leaving pure search cost.

The bottleneck is *solver capability on deeply recursive queries that combine string, integer, bit-vector, and array reasoning*—not anything specific to TOML: `define-funs-rec` is handled by recursive unfolding rather than as a decidable theory, and the parser’s one quantifier relies on MBQI/e-matching, both incomplete heuristics [2, 5]. Difficulty scales with recursion depth and theory combination, not surface grammar size; we present the frontier as a property of *the method×the solver*, illustrated by TOML rather than caused by it. (Four of the 186 markers are *suspected structurally unreachable*, §11.) Two regimes must be separated. *Search* over free input is intractable regardless of encoding: even pinning a prefix while leaving a *single* code point free does not solve, since each free position forces full symbolic recursion through the parser. *Validation* (the input fully determined) is the diagnostic probe. The parser originally represented tables as `Array<String,Value>`, under which even a *fully concrete* candidate returns unknown (“incomplete (theory array)”). We re-encoded tables as an *array-free* inductive assoc-list (`Table = Empty | Bind(key, value, rest)`), replacing the array’s ordering *quantifier* with a recursive minimum-key function—*empirically regression-preserving* (byte-for-byte identical concrete output across both encodings on every unit test and the entire differential corpus; no formal equivalence proof) and decisive: a fully determined candidate (e.g. `a = FALSE` for `boolean_invalid`), pinned as a `define-fun macro` Z3 inlines and constant-folds along the taken branch, now decides *sat* in ~0.42 s where the array encoding (or an `(assert (= input ℓ))` that leaves the input symbolic) returned unknown; the query shrinks only 1.5%, so the win is the *theory*, not size. The consequence is central to §9: search stays out of budget (0/186), *but the recovery loop keeps Z3 in the loop*—a proposer suggests a candidate, Z3 *validates* the macro-inlined input, replay re-certifies—so the solver is the validator, never bypassed.

A deterministic-proposer conformance suite. Independently of synthesis, the reference *is* a conformance oracle: it runs on arbitrary documents, and yields a usable suite whose reach comes from a *fixed, hand-authored candidate table* (`candidates.tsv`), not from search or any model. In the full artifact run, that table covers 182 of the 186 named TOML markers and the validator certifies 151 of them; 35

targets remain uncertified, including the four with no candidate. We use a smaller, human-readable subset for differential testing: *ten* spec-violation markers (an invalid boolean and bare key; the hour/minute/second range checks; an invalid month and day; a malformed float; an invalid string escape; an unterminated inline table). For each, a model-free proposer—one hand-authored document apiece—suggests a candidate, Z3 *validates* the macro-inlined candidate (*sat* in ~0.4 s on the array-free encoding), and concrete replay through the reference parser certifies all *ten* as reaching their named marker, where the solver *search* synthesized *none*. The check is per-target (the `boolean_invalid` document is *rejected* for `date_invalid_month`), the proposer is model-free *by design* (a complete pass over the ten-target subset, not a cherry-picked anecdote), and the certified documents are included as a test—the *direct* route of §9: reach is delegated to an untrusted proposer, acceptance gated by Z3 validation and concrete replay, so the solver is not bypassed.

Answers to the evaluation questions. RQ1 is positive for IMP and STLC: one interpreter serves as oracle and synthesis source, with Z3-synthesized witnesses replay-certified against named markers. It is partially positive for TOML: the same parser is a usable oracle and Z3 validator, but solver-only synthesis does not find witnesses within the reported budget. RQ2 is answered by the double gate: no reported witness is accepted by replay alone or by proposer output alone. RQ3 is the main boundary: IMP solves at native recursion, STLC solves with `k=4` unrolling and a bottom terminator, and TOML separates search from validation (0/186 search witnesses, but sub-second validation of determined candidates after the array-free table encoding). RQ4 is supported by the artifact structure: exact commands, generated SMT, rendered witnesses, replay logs, candidate tables, and summary ledgers are included; the expensive TOML search sweep can be regenerated but is summarized so reviewers can inspect the result without rerunning an hour-long job.

9 AI guides the solver; Z3 validates

A candidate’s acceptance turns on two checks that never consult *what produced* it (§5): Z3 *validates* it against the lifted semantics, and a per-target certifier *certify*(source, *d*) *cross-checks* it by re-parsing and re-executing the concrete reference. The SMT solver is therefore only one *proposer*—a bounded enumerator or a language model is another; all are untrusted. The certifier returns `ReachedTarget`, `ReachedOtherMarker` (with the marker’s *name*), or a rejection (`NoMarker`, `ParseError`, `Timeout`, `Crashed`). Replay runs each candidate in a *separate process* under a wall-clock budget, so an adversarial `while true do skip` is a `Timeout` and a stack overflow a `Crashed`—both reject the candidate, not the run: the arbiter is total over untrusted proposals.

Feature 1: guided synthesis—the model proposes, the solver and replay dispose. The first integration attacks the measured frontier of §8. The solver is asked first; a sat model is decoded and replayed, a native-recursion unsat is recorded as no witness for that encoding, and the proposer enters *only* on the verdicts that carry no semantic conclusion—unknown and timeout. In that loop a configurable proposer (any shell command) sees the brief, the target’s *name*, and earlier verdicts, and emits a *scaffold* of declarative constraints on the input text (prefix/suffix/contains/forbid, char-at-index, ranges, lengths). Each becomes one sequence-theory assertion conjoined to the *unmodified* per-target query, so a scaffold only *strengthens* it: sat decodes and replay-certifies, unsat means the scaffold contradicts the marker (never an unreachability claim), and timeout means “constrain further.” A round is not one model. Because the loop holds a live solver (§4), a sat scaffold is mined for up to `RUSMT_GUIDE_MODELS` *distinct* inputs—each found by blocking the previous one and re-checking—and every one of them is put through both acceptance gates independently. This buys extra chances, never a cheaper acceptance, and it spends milliseconds where another proposer round would spend a model call plus a full solve. Symmetrically, a *stuck* round is not just the bit “timeout”: the surviving process is asked for its own counters (`((get-info :all-statistics))`), so the feedback the proposer receives localizes the stall—recursive-function unfolding versus bit-vector reasoning versus quantifier instantiation—rather than merely reporting that a budget expired. The complementary *direct* route fits the deeply recursive parsers: the model proposes a *whole* candidate, pinned as a `define-fun macro Z3 validates` (sub-second *sat* on the array-free encoding), after which replay re-certifies. Both routes exploit the asymmetry the TOML sweep makes concrete—*searching* the unconstrained query exceeds any budget, yet *validating* a determined candidate is ~0.4 s plus a millisecond replay—and, crucially, *every* accepted witness is *double-gated*: the candidate is re-validated cleanly (macro-inlined, no assumption) and accepted only if that Z3 check is sat *and* replay re-certifies, so no route is replay-only. We exercise the scaffold route end-to-end on a small target with real Z3 and do not claim it extends the TOML frontier; whether current models propose well enough we do not answer (§11), though external evidence suggests feasibility [9, 13, 29].

Feature 2: authoring—proposing and validating the error space. The second integration targets the *error space*, large and easy to under-enumerate by hand. In author mode the human supplies prose grammar/semantics and a condensed DSL reference, and an untrusted model *proposes* the `#[smt_type]/#[smt_fn]` source *with* candidate error-case markers; the framework then *validates* the *declared* markers—we *do not measure* cost reduction, marker recall, or whether the model found the right set (§11). Every draft passes five

gates, each failure fed back for repair. Three are mechanical: the DSL front end (the same parser and sort checker, spanned errors becoming feedback), SMT emission, and at least one named marker. Two are *behavioral*, turning the synthesis machinery (§6) on the draft. *Marker reachability*: each marker’s query is built from the draft’s IR and solved at $k=0$, so unsat is a solver-*proved* dead-code fact, sat records the witness, a timeout is a warning. *Spec examples*: human expectations (input paired with `ok`, `err:<marker>`, or `nomatch`) are each fixed by one equality assertion and the solver checks which markers the result carries, so a mismatch is precise feedback; examples come from the spec or its author, never the model. The gates establish admissibility plus these behavioral facts—not semantic fidelity, established as for any hand-written spec (review, synthesis, differential testing). Totality (§3) means no *handled* error is silently swallowed, but it does not force the model to *discover* every spec-implied violation (in one trial a model’s DSL draft dropped a second-range check its plain-Rust draft kept), so the enumeration is not *provably* complete; gate (v) surfaces such omissions (`00:00:99 → err:second_out_of_range` fails a forgetful draft) and differential testing catches the rest. The result is *the declared markers, validated for reachability and example-conformance*—stronger than a chatbot’s unchecked list, weaker than a completeness proof, and weaker than machine-checked-proof gates for LLM code in verification-aware languages [15, 25]: an executable semantics has no independent spec to verify against (it *is* the spec). That models track syntactic priors over semantics [26] is why the model never arbitrates its own draft.

What is and is not claimed. Both integrations are implemented, vendor-agnostic (the proposer is an arbitrary command—no model, API, or key in the pipeline), exercised by scripted proposers plus fast real-Z3 tests, and off by default. We claim the architecture and its soundness discipline—acceptance is decided by the solver and replay, or by the gates, never by the model—but *no* reach gain or authoring-effort reduction for any particular *model* (§11). The discipline extends to a model guiding search *inside* Z3 via lemmas it checks [13, 32]—future work.

10 Related work

Solver-aided executable semantics. Rosette [27, 28] is the closest solver-aided host: a symbolic VM compiles a language subset to constraints while concrete evaluation strips away ordinary host code. RuSMT is narrower and more conformance-oriented: the user writes a reference interpreter, marks specification obligations, and receives tests accepted only by Z3 validation plus replay. SemGuS [10] synthesizes from syntax and semantics encoded as constrained Horn clauses; RuSMT instead derives formulas from ordinary recursive functions. Synantic [12] synthesizes a semantics

from a black-box interpreter, complementary to our source-to-SMT route.

Specification-derived testing. Executable-semantics frameworks such as K [20], PLT Redex [7], and Ot-t/Lem [16, 24] derive tools from rewrite or inference-rule specifications. RuSMT derives a more focused tool from direct-style Rust: marker-directed conformance tests. JEST [17] is closest in purpose, using a mechanized ECMAScript spec as oracle, synthesizer, and conformance tester. Our distinction is the Z3/replay acceptance gate on author-named markers and the explicit sound-but-incomplete split exposed by the TOML case study. Coverage- and grammar-driven generators such as Csmith [33] remain the application domain rather than the technique: RuSMT derives targeted witnesses from the executable specification.

Language models behind trusted checkers. We do not claim novelty for the proposer/checker pattern. Lemur [32] uses LLM-proposed verification structure, and AquaForte [13] has an LLM propose SMT models that the solver rechecks. Test-generation portfolios such as CodaMOSA [11] and Cottontail [29] similarly call a model only when conventional generation stalls. RuSMT’s modest difference is where the checker lives: the same executable semantics is oracle, synthesis source, and replay certifier, and the accepted obligation is a stable named marker rather than generic coverage. The authoring mode follows the same principle: an LLM may draft source, but the framework admits it only through the front end, backend, marker reachability, and human-supplied examples.

11 Limitations and threats to validity

Scope and subset. The DSL excludes loops, mutation, and references; the price buys a clean SMT translation, mirrors textbook big-step semantics, and is mechanically enforced. RuSMT targets a specification’s *negative space*—its syntactic and semantic error branches—not happy-path fuzzing (§1). A marker covers only an error condition the author wrote; a violation the specification implies but the reference omits is not marked (the authoring mode’s spec examples, §9, mitigate but do not close this). The TOML sweep used a reduced 20 s budget, not the full 600 s sweep. Naming TOML markers does not change the lifted query, only which targets are name-addressable. The STLC case study is a bounded-synthesis *demonstration*, not evidence of generalization: a small hand-written checker (eight markers, $k=4$ only, no variance study, and—unlike IMP and TOML—no external implementation for differential comparison); it shows the loop closes via the bottom-terminator discipline, not that it scales.

The authoring burden. The artifact is *write-once, use-many*—one source is oracle, synthesis source, and arbiter,

amortizing over a reusable specification rather than a throw-away generator [7, 20]. The burden is *lowered but not removed* by AI drafting (author mode, §9): the model proposes candidate markers (the framework validates only the *declared* ones; it does not establish completeness), but a human must choose the DSL-expressible subset, supply the spec examples, and review and own the result—neither model nor gates certifies fidelity to the prose. The approach thus pays off when a reusable reference semantics is wanted anyway, and is poor economics for a one-off test.

Soundness validation and the solver frontier. The per-construct contract is *checked* only for the bit-vector and string/sequence families (§8), not the 32-bit variants, reals, or floats, where a divergent primitive would silently degrade faithfulness; extending the harness is the most important remaining validation work (documented formulas and example tests are the interim defence). We claim only *bounded* completeness, contingent on the solver. The dominant limitation is solver capability on deeply recursive, multi-theory queries: even after the array-free re-encoding restores concrete-candidate *validation*, free-input *search* stays out of budget (0/186), so a proposer, not the solver alone, reaches a witness on TOML; pushing search itself is the main open problem. A natural next step lets a stuck solver hand control back through Z3’s user-propagator API—callbacks at decision points the solver controls, no vendored patch—so a proposer could inject solver-*rechecked* lemmas [13, 32], or, more ambitiously, register the concrete reference semantics itself as a propagated theory so that a candidate model is replayed *inside* the search and a rejection returns a conflict clause rather than merely discarding a round. Notably this does not require a second, hand-built encoder: `Z3_solver_from_string` loads the backend’s own emitted SMT-LIB into an in-memory solver, to which a propagator attaches directly—so the inspectable-text pipeline is a front end to in-process experimentation, not an obstacle to it.

Scope of the AI integrations. The two AI features are architecture, not measured *model* capability: *no language model was invoked for any reported result*. All TOML recovery evidence uses a deterministic, model-free scripted proposer—a fixed lookup over an artifact-included candidate table—standing in for any untrusted proposer; we report no LLM recovery rate, model comparison, or authoring-effort measurement, which is future work. The TOML denominators are distinct and must not be conflated: the pure *search* sweep is 0/186, while a separate scripted-proposer run certifies 151/186 (the candidate map covers 182/186). Of the four markers with no candidate, all four *appear structurally unreachable*, so the honest denominator is 182 reachable-or-hard plus 4 suspected-dead. The authoring gates establish admissibility and checked behavioral facts, not fidelity—trust

in a draft is bounded by downstream validation, as for any hand-written specification.

Threats to validity, by category. Read against the standard empirical-software-engineering taxonomy [21, 31]: *construct*—a marker measures an error the reference author wrote, not the specification’s true error space, and the per-construct contract is checked only for the bit-vector and string/sequence families (2715+5317+216 Z3-backed checks), so “the lift is faithful” is measured there and argued for reals, floats, and 32-bit vectors. *Internal*—acceptance is by replay under *named* markers whose id coincides on both sides by construction, so a witness is credited only when re-execution fires *that* target, excluding “some-marker” confounds and spurious k-bounded models (§8); the array-free re-encoding is controlled by byte-identical concrete output across both encodings on every test (regression-preserving, not formally proven equivalent). *External*—three case studies, one machine, one solver version, no cross-machine claim; the frontier is exhibited by TOML but *argued* from recursion depth $\times$ theory combination to be a property of method $\times$ solver. *Conclusion*—all timings and the 0/186 sweep are a single run, and Z3’s randomized heuristics make latencies run-dependent, so we report the qualitative regime (solves quickly vs. times out), pin the random seed, and content-address each query; “zero disagreement” attests no counterexample *within* the checks, not exhaustive equivalence.

12 Conclusion

RuSMT lets an author write a language’s reference semantics once, as an ordinary recursive Rust program that is at the same time the conformance oracle and the source of the tests. From it an SMT solver *proves* which programs reach the spec’s named error markers, and concrete re-execution certifies each accepted witness, so—unlike a fuzzer—no human inspects a test to trust it. The one obligation this rests on, that the lift is faithful to the Rust it transpiles, we check per primitive rather than assume. On that base we integrate AI twice without trusting it: on the solver’s unknown/timeout frontier an untrusted proposer reaches while Z3 validates and replay re-certifies (an array-free re-encoding makes that validation sub-second where it was unknown), and a gated authoring mode aims at the cost of writing the specification itself. We offer RuSMT as a template for conformance testing from living, executable specifications, with AI a proposer that guides the solver and never a judge.

References

- [1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-Guided Synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*. IEEE, 1–8.
- [2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6*. Technical Report. Department of Computer

- Science, The University of Iowa. Available at <https://smt-lib.org>.
- [3] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS, Vol. 1579)*. Springer, 193–207. doi:10.1007/3-540-49059-0_14
- [4] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 114–129. doi:10.1109/SP.2014.15
- [5] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [6] Ecma International TC39. [n.d.]. Test262: ECMAScript Conformance Test Suite. <https://github.com/tc39/test262>. Accessed 2026-06-20.
- [7] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex*. MIT Press.
- [8] Haryadi S. Gunawi, Cindy Rubio-González, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Ben Liblit. 2008. EIO: Error Handling is Occasionally Correct. In *6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 207–222.
- [9] Sumit Kumar Jha, Susmit Jha, Patrick Lincoln, Nathaniel D. Bastian, Alvaro Velasquez, Rickard Ewetz, and Sandeep Neema. 2023. Counterexample Guided Inductive Synthesis Using Large Language Models and Satisfiability Solving. In *IEEE Military Communications Conference (MILCOM 2023)*. IEEE. doi:10.1109/MILCOM58377.2023.10356332
- [10] Jinwoo Kim, Qinheping Hu, Loris D’Antoni, and Thomas W. Reps. 2021. Semantics-Guided Synthesis. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–32. doi:10.1145/3434311
- [11] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Sid-dhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering (ICSE 2023)*. IEEE. doi:10.1109/ICSE48619.2023.00085
- [12] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D’Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 362–388. doi:10.1145/3689724
- [13] Kunhang Lv, Yuhang Dong, Rui Han, Fuqi Jia, Feifei Ma, and Jian Zhang. 2026. LLM-Guided Quantified SMT Solving over Uninterpreted Functions. In *Proc. 40th AAAI Conference on Artificial Intelligence*. arXiv:2601.04675.
- [14] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [15] Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. 2024. Towards AI-Assisted Synthesis of Verified Dafny Methods. *Proc. ACM Softw. Eng. (FSE)* 1 (2024). doi:10.1145/3643763
- [16] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: Reusable Engineering of Real-World Semantics. In *Proceedings of the 19th SIGPLAN International Conference on Functional Programming (ICFP 2014)*. ACM, 175–188. doi:10.1145/2628136.2628143
- [17] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*. IEEE, 13–24. doi:10.1109/ICSE43902.2021.00015
- [18] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press.
- [19] Tom Preston-Werner et al. 2025. TOML: Tom’s Obvious, Minimal Language, Version 1.1.0. <https://tomli.io/en/v1.1.0>. Language specification.

- [20] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012
- [21] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8
- [22] Len Sassaman, Meredith L. Patterson, Sergey Bratus, and Michael E. Locasto. 2013. Security Applications of Formal Language Theory. *IEEE Systems Journal* 7, 3 (2013), 489–500. doi:10.1109/JSYST.2012.2222000
- [23] Nicolas Seriot. 2016. Parsing JSON is a Minefield. http://seriot.ch/parsing_json.php. Accessed 2026-06-20.
- [24] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: Effective Tool Support for the Working Semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*. ACM, 1–12. doi:10.1145/1291151.1291155
- [25] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. In *AI Verification (SAIV 2024) (Lecture Notes in Computer Science)*. Springer. arXiv:2310.17807
- [26] Aditya Thimmaiah, Jiyang Zhang, Jayanth Srinivasa, Junyi Jessy Li, and Milos Gligoric. 2025. PLSemanticsBench: Large Language Models as Programming Language Interpreters. arXiv:2510.03415
- [27] Emina Torlak and Rastislav Bodík. 2013. Growing Solver-Aided Languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*. ACM, 135–152. doi:10.1145/2509578.2509586
- [28] Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. ACM, 530–541. doi:10.1145/2594291.2594340
- [29] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. 2025. Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation. *arXiv preprint arXiv:2504.17542* (2025). arXiv:2504.17542 To appear, IEEE Symposium on Security and Privacy (S&P) 2026.
- [30] Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press.
- [31] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. doi:10.1007/978-3-642-29044-2
- [32] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.04870
- [33] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. ACM, 283–294. doi:10.1145/1993498.1993532
- [34] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. 2014. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 249–265.